

Transformers

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

The Gap Transformers Fill

A direct continuation of the RNN limitation from the last session



RNNs (recap)

Process one timestep at a time. Can't parallelize across the sequence, and gradients decay over long distances — even with LSTM/GRU gating.



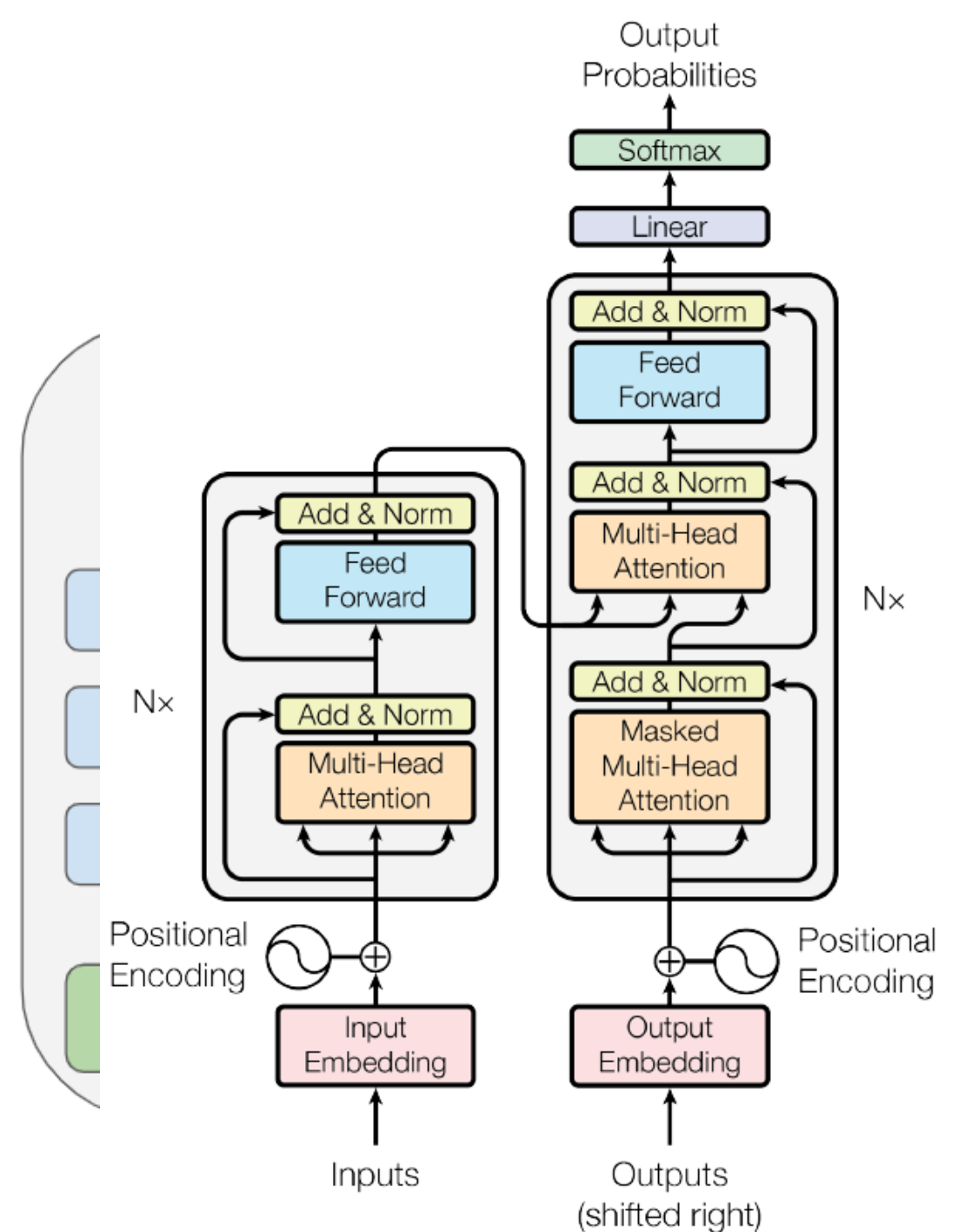
Transformers

Any position can directly attend to any other position, regardless of distance — and every position can be computed in parallel during training.

“Attention Is All You Need” (2017) — the paper that dropped recurrence entirely and built a model on attention alone.

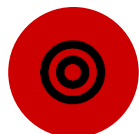
Overview of Architecture

- Transformers are encoder/decoder architectures that were introduced for sequence-to-sequence tasks in NLP.
- The input is given by $X = \{x_1, x_2, \dots, x_T\}$ and the output is $\{y_1, y_2, \dots, y_K\}$.
- In the diagram, the shifted output is what has been decoded produced so far.
- The architecture aims to model $p(y_t | y_{1:t-1}, X)$



Self-Attention: The Core Idea

Every position looks at every other position, and decides how relevant each one is



Weighing Relevance

Each word computes a relevance score against every other word in the sequence — not just its immediate neighbors.



Computed Fresh, Every Time

The weights aren't fixed rules — they're computed dynamically based on the specific input in front of the model.



Direct, Not Chained

Word 1 and word 50 can relate directly in a single step — no need to pass information through 49 intermediate steps.

In short: attention lets the model ask, for every word, “which other words actually matter for understanding this one — right now?”

Self-Attention: A Worked Example

The classic case — figuring out what “it” refers to



Self-attention lets the model weigh “animal” far more heavily than “street” when interpreting “it” — using the surrounding context, not word order or distance.

Swap “tired” for “wide” and the weights flip: “it” now points to “street.” Same mechanism, different input, different answer.

Self-Attention: The Mechanism (Query, Key, Value)

How the weighting in the previous example actually gets computed



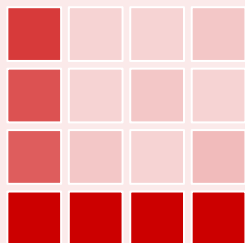
A useful analogy: Query = what “it” is looking for. Key = what each word advertises about itself. Value = what that word actually contributes once attended to — like a search engine matching a query against document titles, then returning the document contents.

This whole sequence runs once, in parallel, for every token — that parallelism is exactly what RNNs couldn't offer.

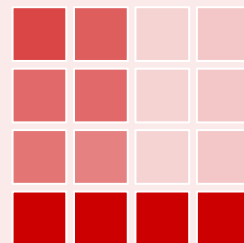
Multi-Head Attention: Several Views at Once

Run attention several times in parallel, then combine the results

Head 1: Coreference



Head 2: Local Syntax



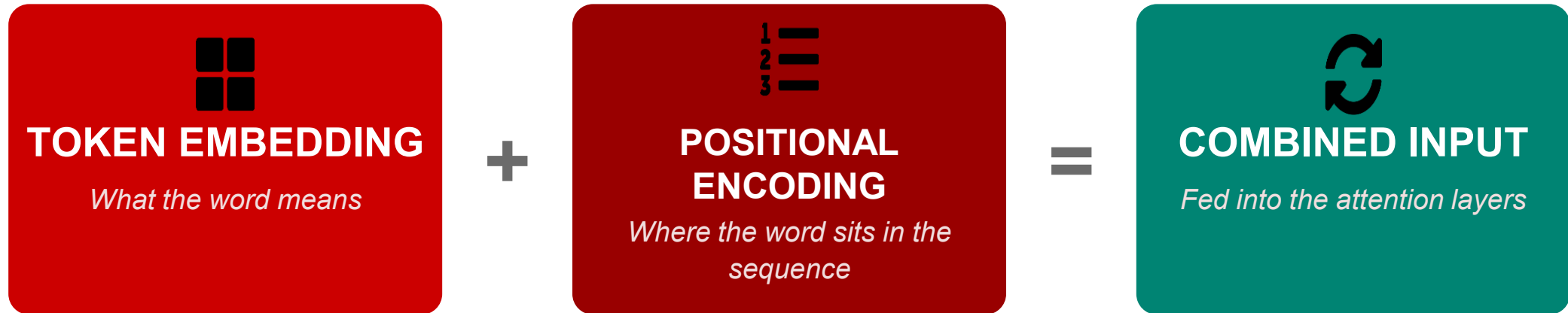
Head 3: Broad Topic



The outputs of all heads are concatenated and combined — one head might specialize in coreference, another in local word order, another in broad topic.

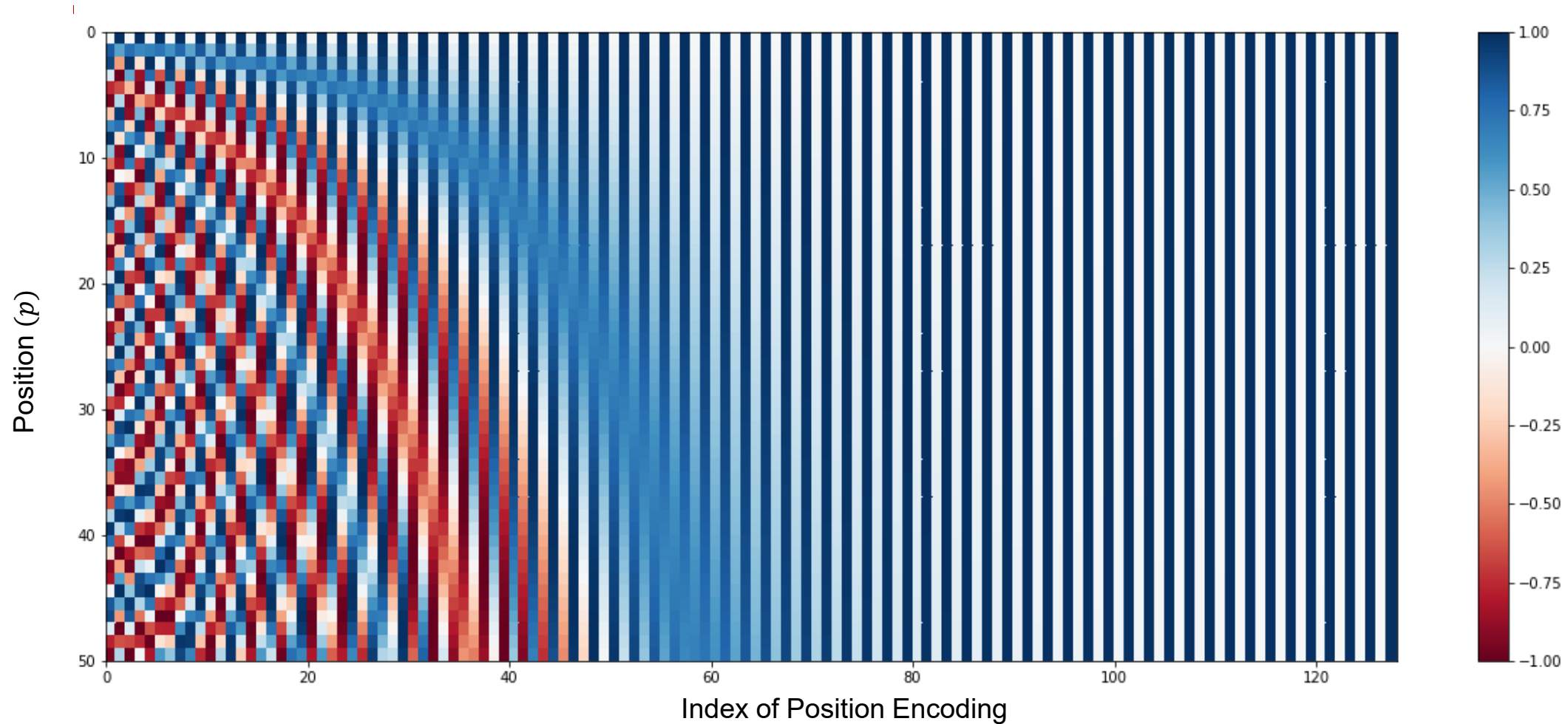
Positional Encoding: Giving Attention a Sense of Order

Attention alone has no idea which word came first



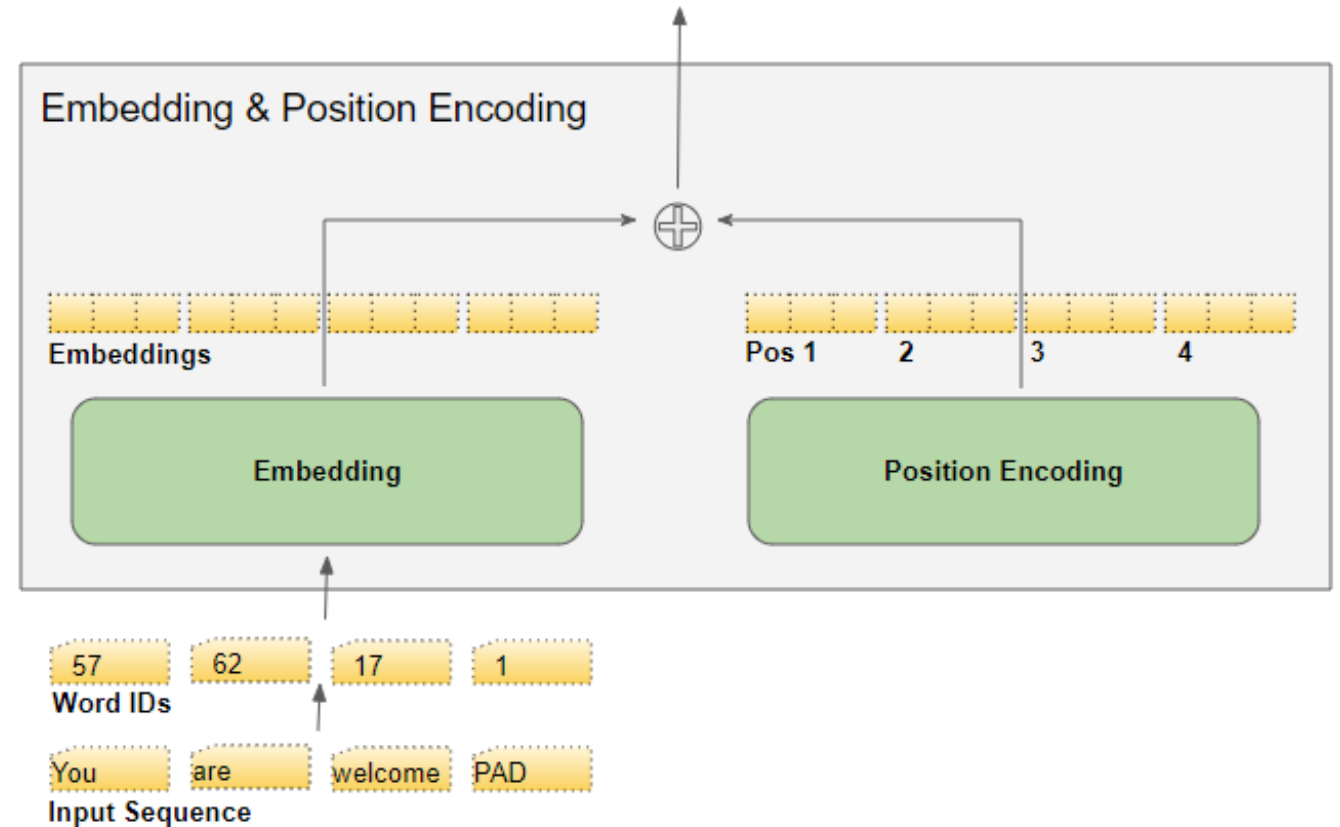
Unlike an RNN, attention treats the input as a set, not a sequence — without this addition, “dog bites man” and “man bites dog” would look identical to it.

Positional Encoding



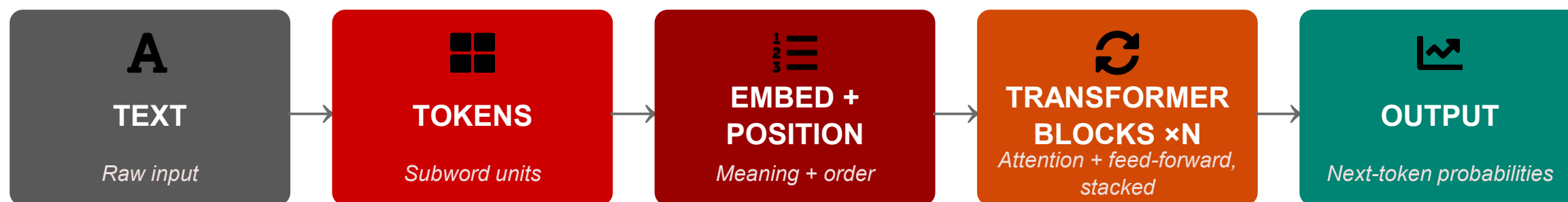
Embedding

- A **token** is a numeric value assigned to a word or part of it.
- An **embedding** takes the token and maps it to a vector in $\mathbb{R}^{d_m}$ for a large d_m by ensuring that their dot product corresponds to some type of relation. This is essentially a look-up table.
- The embedding can be learned as part of the model or a pre-trained embedding can be used, e.g., GloVe.



How Transformers Process Text, End to End

From raw text to a prediction — the full pipeline



This entire pipeline is what runs once per step of the autoregressive loop from the LLM session — same pipeline, run again on the longer sequence each time.

The Scale of State-of-the-Art Models

The parameter counts behind this architecture have grown by orders of magnitude

GPT-2 (2019)

1.5B

parameters

GPT-3 (2020)

175B

parameters — ~117× GPT-2

Open MoE models (2026)

1T+

*total parameters (e.g., trillion-parameter
mixture-of-experts models)*

Important nuance: in a mixture-of-experts model, only a fraction of those parameters are “active” for any given token — a trillion-parameter model isn’t doing a trillion-parameter computation on every word.

Most current frontier labs (OpenAI, Anthropic, Google) no longer disclose exact parameter counts — open-weight models are now the best public window into current scale.

Diffusion

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

The Gap Diffusion Models Fill

Two earlier approaches to image generation, and their tradeoffs



GANs & Plain VAEs

GANs pit two networks against each other — famously unstable to train. Plain VAEs train stably but tend to produce blurry, averaged-out images.



Diffusion Models

A simple, stable training objective — predict the noise — with no adversary to destabilize it. The cost: generation takes many small steps instead of one shot.

Recall from the autoencoder session: a VAE that samples its latent space is already a generative model — diffusion is the far more capable descendant of that same idea.

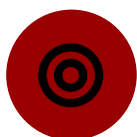
The Core Idea: Learn to Reverse a Corruption Process

Training is destruction. Generation is undoing it, one small step at a time.



Corrupt, Then Learn to Reverse

Training adds noise to real images in a known, fixed way — the network only has to learn to undo that specific, well-understood process.



No Adversary Needed

Unlike a GAN, there's no second network to fight against — just a direct reconstruction-style loss, the same flavor as an autoencoder's.



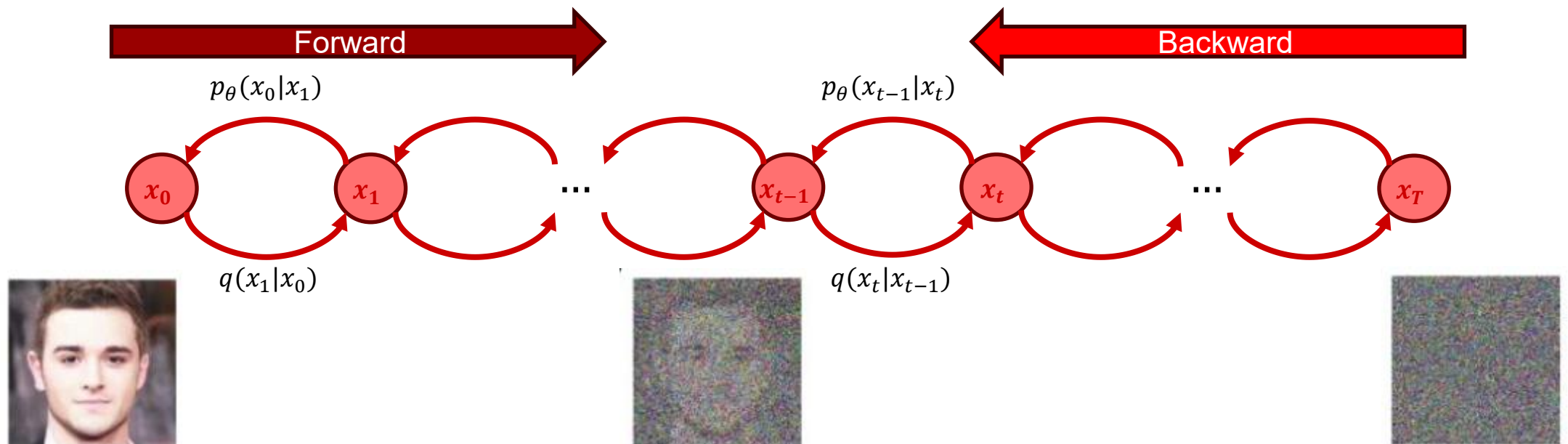
Many Small Steps, Not One Big Leap

Generation refines a noisy guess gradually over dozens of steps, rather than committing to a final image all at once.

Everything that follows is really just: how is the corruption applied, and how does the network learn to reverse it?

Motivation

- For Diffusion, we consider a stochastic Markovian process in which we are aiming to obtain samples from the real image distribution.
- A simple **forward process** is defined in which we inject Gaussian noise to the states until a Gaussian distribution is obtained.
- The **backward process** reserves each steps aiming to map from a noisy distribution to the original data manifold.



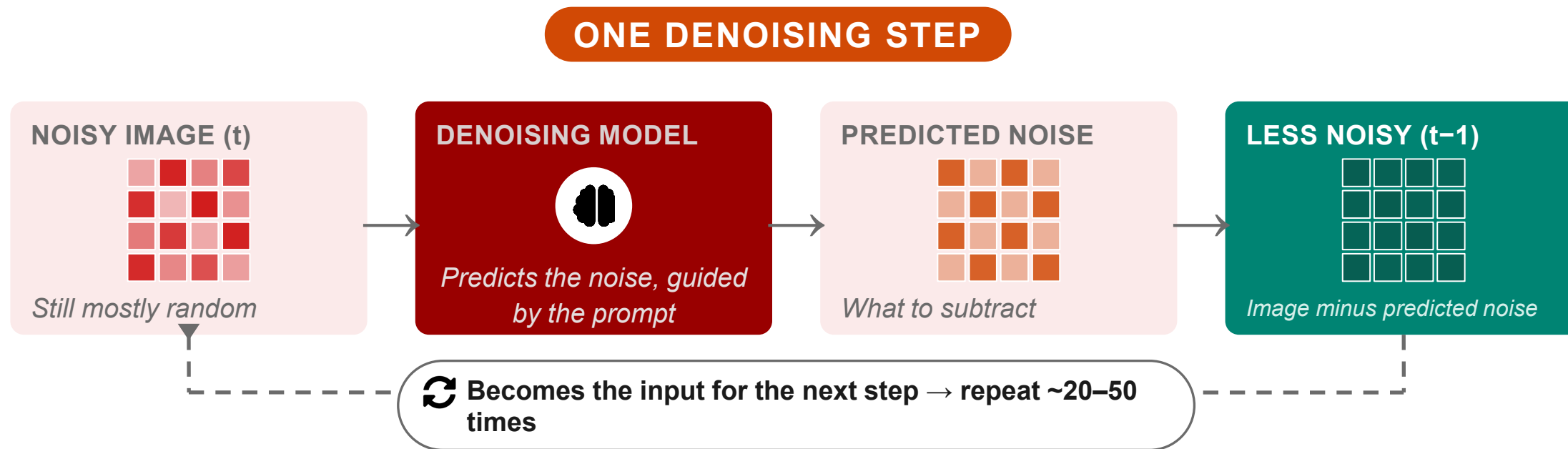
Diffusion Process

- Illustration of Diffusion Process:



Reverse Diffusion: One Step, Repeated

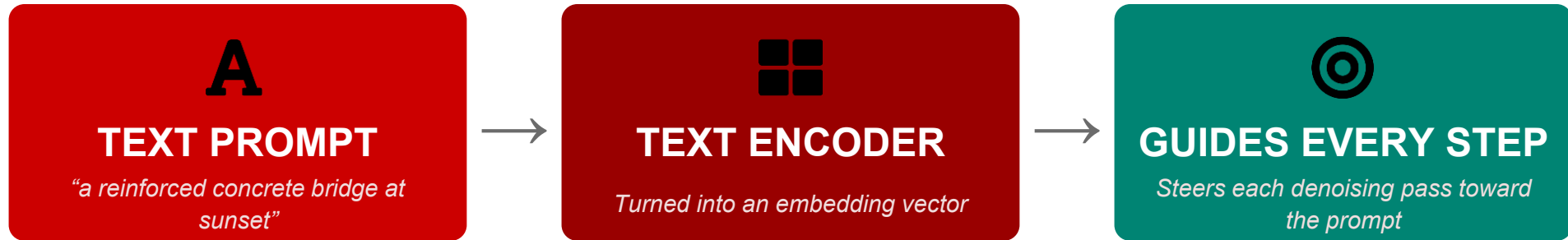
The same recurrent-loop pattern from the LLM and CNN/RNN sessions



The model itself is stateless per call — all progress lives in the image tensor passed between steps, exactly like the token sequence in text generation.

Text Conditioning & Guidance

The same prompt steers every single denoising step



Classifier-free guidance controls how strongly the image follows the prompt. Too low and the result drifts off-topic; too high and it becomes oversaturated and artifact-heavy — it's a tunable dial, not a fixed setting.

This is exactly the prompt bar from the LLM session's image-generation slide — same idea, now with the mechanics behind it.

Scale & Applications of State-of-the-Art Models

The parameter counts — and the architecture shift underneath them

SD 1.5 (2022)

0.86B

SDXL (2023)

~2.6B

SD 3.5 / Flux (2024–26)

8–12B

Architecture shift worth noting: the newest models (SD 3.5, Flux) replace the U-Net with a Diffusion Transformer (DiT) — the same attention mechanism from the previous session, now generating images instead of text.

Engineering applications: generative design visualization, and synthetic training images for rare failure modes — both raised back in the classical-ML transition session.